

PHP | Decision Making

- Decision-making is an important part of programming, allowing the program to execute different actions based on conditions.
 - In PHP, decision-making helps control the flow of a program by executing different blocks of code depending on certain conditions or expressions.
 - PHP provides several constructs for decision-making, including if, else, elseif, and switch. These control structures can be used to make logical decisions in a program.
-

1. if Statement

The if statement is the simplest form of decision making. It executes a block of code if the specified condition evaluates to true. If the condition evaluates to false, the block of code is skipped.

Syntax:

```
if (condition) {  
    // code to be executed if condition is true;  
}
```

Example 1:

```
<?php  
$age = 20;  
if ($age >= 18) {  
    echo "You are eligible to vote."  
}  
?>
```

Output:

You are eligible to vote.

Example 2:

```
<?php  
$num=12;  
if($num<100) {  
    echo "$num is less than 100";  
}  
?>
```

Output:

12 is less than 100

2. if...else Statement

The if-else statement is an extension of the if statement. It allows you to specify an alternative block of code that is executed when the condition is false. This is useful when there are two mutually exclusive conditions.

Syntax:

```
if (condition) {  
    // code to be executed if condition is true;  
} else {  
    // code to be executed if condition is false;  
}
```

Example 1:

```
<?php  
$num=12;  
if($num%2==0){  
    echo "$num is even number";  
}else{  
    echo "$num is odd number";  
}  
?>
```

Output:

12 is even number

Example 2:

```
<?php  
$age = 16;  
if ($age >= 18) {  
    echo "You are eligible to vote.";  
} else {
```

```
    echo "You are not eligible to vote.";
}
?>
```

Output:

You are not eligible to vote.

3. if...elseif...else Statement

In scenarios where you need to evaluate multiple conditions, you can use the if-elseif-else ladder. This construct allows you to check multiple conditions in sequence. The first condition that evaluates to true will execute its corresponding block of code, and all other conditions will be skipped.

Syntax:

```
if (condition) {
    // code to be executed if this condition is true;
} elseif (condition) {
    // code to be executed if first condition is false and this condition is true;
} else {
    // code to be executed if all conditions are false;
}
```

Example 1:

```
<?php
$month = "August";
if ($month == "January") {
    echo "Happy New Year!";
} elseif ($month == "August") {
    echo "Happy Independence Day!";
} else {
    echo "No special occasion.";
}
?>
```

Output:

Happy Independence Day!

4. Switch Statement

The "switch" performs in various cases i.e., it has various cases to which it matches the condition and appropriately executes a particular case block. It first evaluates an expression and then compares with the values of each case. If a case matches then the same case is executed. To use switch, we need to get familiar with two different keywords namely, break and default.

Syntax:

```
switch(expression) {  
  case value1:  
    //code to be executed  
    break;  
  case value2:  
    //code to be executed  
    break;  
  .....  
  default:  
    code to be executed if all cases are not matched;  
}
```

Example 1:

```
<?php  
$num=20;  
switch($num) {  
  case 10:  
    echo("Number is equals to 10");  
    break;  
  case 20:  
    echo("Number is equal to 20");  
    break;  
  case 30:  
    echo("Number is equal to 30");
```

```
break;  
default:  
echo("Number is not equal to 10, 20 or 30");  
}  
?>
```

Output:

Number is equal to 20
